Open-Ended Project Write-Up

1 Design Approach

The final project implements a hardware-based image processing system using MATLAB and SystemVerilog. A PNG image is input into MATLAB, converted to grayscale, and resized to be 256x256 pixels. MATLAB outputs the image into a text file as 8-bit pixel values, which will be used as input for the hardware modules in Quartus.

Three different hardware designs were created in Quartus. The first one is a sequential design that processes one pixel per clock cycle and performs a brightness adjustment. Second is a parallel design that performs the same operations. However, it processes four pixels per clock cycle. The third is a Sobel design, which performs edge detection using 3x3 neighbor pixels, requiring a more complex computation due to its unique architecture.

After all designs are processed, the output pixel values are written to a file and imported back into MATLAB to create a PNG image, allowing for comparison between input and output images.

Figure 1: Moon image

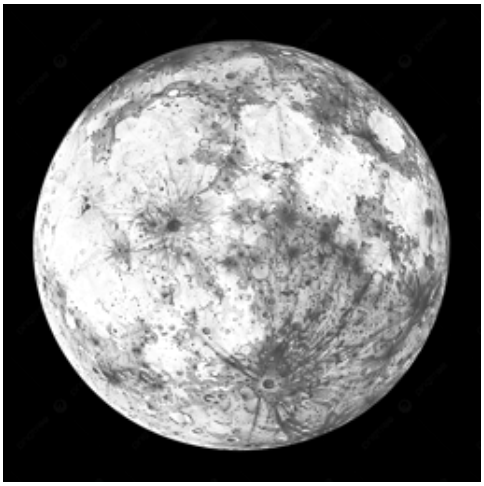

Figure 2: MATLAB output image

2 Architecture Trade-offs

The three different design choices display different trade-offs such as simplicity, performance, and computational complexity.

The sequential design is the simplest and uses minimal hardware resources. However, it is the slowest option because it requires one clock cycle per pixel.

The parallel design improves performance by processing four pixels per clock cycle. This reduces the total number of cycles required, increasing throughput. The trade-off is that more operations are performed per cycle, which would increase hardware usage and dynamic power. However, synthesis results indicate lower resource usage for the parallel design (see Section 4).

The Sobel design introduces a more complex algorithm. Instead of operating on a single pixel, it uses a 3×3 neighborhood to calculate gradients and detect edges. This increases computational complexity and data dependency but provides a more unique image processing result.

3 Implementation Challenges

One of the main challenges was managing the data flow between MATLAB and the hardware. This included formatting pixel values into hexadecimal text files and checking that the dimensions matched between MATLAB and SystemVerilog.

Another challenge was handling image boundaries in the Sobel design. Since the Sobel filter requires neighboring pixels, special handling was needed for pixels at the edges of the image to avoid invalid memory access.

There were also challenges related to MATLAB visualization. The MATLAB online environment would often time out while attempting to display the image, so the output images were saved as PNG files instead for verification.

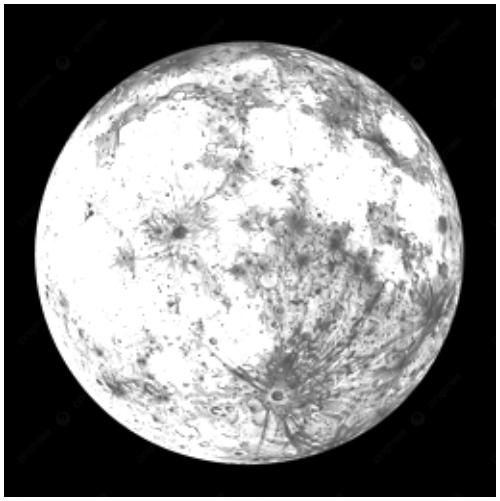

Figure 3: Sequential output image

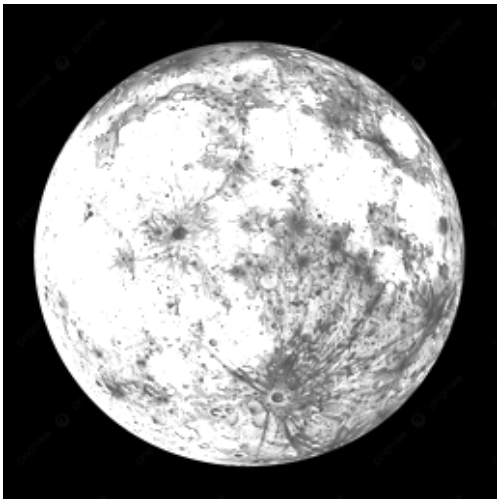

Figure 4: Parallel output image

Figure 5: Sobel output image (copyright is lightly shown)

Just to reiterate, the RTL implementation uses register-based storage and sequential control logic to read and process pixel data. The sequential design uses a single processing unit that operates once per clock cycle. The parallel design replicates the processing logic across four data paths, allowing multiple pixels to be processed simultaneously. The Sobel design requires additional buffering to access neighboring pixels, increasing control and data movement complexity.

4 Performance Analysis

The table below summarizes the hardware performance of each design:

Metric	Sequential	Parallel	Sobel
Logic Utilization (in ALMS)	15/113,560 (<1%)	9/113,560 (<1%)	15/113,560 (<1%)
Total Registers	18	15	18
Fmax	349.41 MHz	284.17 MHz	349.41 MHz
Total Thermal Power Dissipation	526.60 mW	526.60 mW	526.60 mW
Core Static Thermal Power Dissipation	520.91 mW	520.91 mW	520.91 mW
I/O Thermal Power Dissipation	5.70 mW	5.70 mW	5.70 mW

The sequential and Sobel designs both achieve a higher Fmax (~349.41 MHz), while the parallel design has a lower Fmax (~284.17 MHz). All three designs use very small hardware resources (less than 1% ALMs), so resource usage is not a good way to differentiate between designs.

All designs show similar thermal power dissipation, core static power dissipation, and I/O thermal power dissipation due to designs being too small to show dynamic differences.

The sequential and Sobel designs require 65,536 cycles to process the full 256x256 image, since they operate on one pixel per cycle. The parallel design reduces this to 16,384 cycles by processing four pixels per cycle, resulting in a 4x improvement in throughput.

```
# Sequential cycles = 65536
# Parallel cycles   = 16384
# Sobel cycles      = 65536
```

The Sobel design runs at a similar frequency to the sequential design, but it performs more operations per pixel due to the 3×3 calculation. So even though the cycle count is similar, the computation being done is more complex.

Overall, the results show that good performance is mainly determined by the total number of cycles.

5 Verification

Verification was performed by comparing the reconstructed output images in MATLAB. The input image generated in MATLAB was used as a reference. The sequential and parallel outputs were compared against the MATLAB-generated image to check correctness and confirmed to produce similar brightness-enhanced images. This confirms functional correctness, since both designs produce identical pixel transformations for the same input.

The Sobel output was also verified by observing edge detection in the reconstructed image. The output clearly highlights boundaries and structural features, confirming correct implementation of the Sobel filter. An interesting observation is that the Sobel output shows the watermark more clearly than the other images, showing how accurate the edge detection is.

6 Conclusion

This project demonstrates how image processing algorithms can be implemented in hardware and evaluated using different architectural approaches. The sequential and parallel designs

show how performance can be improved through parallelism, while the Sobel design demonstrates a more complex computation using neighboring pixel data.

Although the synthesis results show similar hardware metrics, the designs differ in throughput and computational complexity. The project successfully meets the requirement by including both non-trivial computation and control flow.

Overall, this work highlights the relationship between algorithm complexity, hardware architecture, and performance in VLSI system design.